

Department of Computer Science and Engineering Assignment Questions

Course Code & Name : CSA0609 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E-Computer Science and Engineering
Course Code & Course Name	CSA0609-Design and Analysis of Algorithms
Academic Year / Batch	2026-Regular
Faculty Name	Dr.P.Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

B. Assignment Problem / Challenge

Given a set of 10 two-dimensional points,

$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$

the objective of this assignment is to conduct an **analytical study and practical implementation of Brute Force algorithms** for solving the **Closest Pair** and **Convex Hull** problems.

For the **Closest Pair Problem**, apply the Brute Force technique by systematically examining every possible pair of points in the given two-dimensional dataset. For each pair, calculate the Euclidean distance using the standard distance formula and compare the calculated distances to determine the pair of points having the minimum distance. The analysis should clearly show the major distance calculations, identify the closest pair, and report the corresponding minimum Euclidean distance.

For $n=10$ points, determine the total number of unique pairwise distance comparisons using the combination formula:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

and explain how the number of comparisons increases as the number of input points increases. Analyze the **time complexity and space complexity** of the Brute Force Closest Pair algorithm and evaluate its scalability for larger datasets.

For the **Convex Hull Problem**, apply the Brute Force technique to the same set of 10 two-dimensional points. Examine all possible pairs of points as candidate edges and use an orientation test to determine whether the remaining points lie on the same side of the line formed by each candidate pair. Candidate edges satisfying the required condition are identified as boundary edges of the convex hull.

The orientation of three points should be determined using the cross-product/orientation formula. The analysis should include the important intermediate orientation calculations, identification of valid candidate edges, ordering of the resulting hull vertices, and determination of the final convex hull of the given point set.

The assignment should determine the **number of candidate edges** considered by the Brute Force Convex Hull approach. For $n=10$, the number of possible pairs of points is:

$$\binom{10}{2} = \frac{10(9)}{2} = 45$$

Therefore, up to **45 candidate edges** may be examined. For each candidate edge, the algorithm may need to examine the remaining points to determine whether the edge can belong to the convex hull. Based on this procedure, analyze and derive the **time complexity and space complexity** of the Brute Force Convex Hull algorithm.

A suitable **algorithm and pseudocode** shall be developed for both the Closest Pair and Convex Hull problems. The algorithms shall then be implemented using **Python**, preferably in a Google Colab environment. The implementation should display the input points, calculated results and graphical representation of the final Convex Hull. The Closest Pair should also be visually highlighted in the coordinate plane.

To evaluate the practical performance of the algorithms, experiments should be performed using increasing input sizes. The execution time, number of comparisons, number of orientation operations and other relevant performance measures should be recorded. The experimental results should be presented using appropriate **tables, graphs and visualizations**.

The study should further analyze how the Brute Force approach behaves as n increases and compare the theoretical complexity with the experimentally observed performance. Where appropriate, the

Brute Force solutions may be compared with more efficient approaches such as **Divide-and-Conquer for Closest Pair** and **Graham Scan or other efficient Convex Hull algorithms**.

The final assignment should include the **problem formulation, mathematical foundations, algorithm design, pseudocode, implementation, step-by-step calculations, graphical representation, experimental results, complexity analysis, comparative evaluation, applications, limitations, conclusion and future improvements**.

Primary objectives of the challenge are:

- To understand the Brute Force algorithmic paradigm.
- To formulate the Closest Pair problem mathematically.
- To formulate the Convex Hull problem mathematically.
- To calculate the Euclidean distance between every possible pair of points.
- To identify the closest pair among the given points.
- To determine the number of pairwise comparisons for $n=10$.
- To examine all possible candidate edges for the Convex Hull.
- To apply the orientation test to identify valid hull edges.
- To determine the points belonging to the Convex Hull.
- To develop algorithms and pseudocode for both problems.
- To implement the algorithms using Python.
- To visualize the input points, closest pair and resulting Convex Hull.
- To analyze the time and space complexity of the algorithms.
- To experimentally evaluate the performance of the algorithms for increasing input sizes.
- To compare the performance of the Brute Force algorithms with more efficient algorithmic approaches.
- To identify the advantages, limitations and practical applications of the studied algorithms.

C. Requirements and Constraints

Yes. For **your title** and the exact structure of your reference document, the **Requirements and Constraints** should focus specifically on the **Brute Force Closest Pair and Convex Hull problems**. Also, I'll keep the numbering in a clean continuous format.

B. Requirements and Constraints

The assignment shall be carried out using the given set of 10 two-dimensional coordinate points and suitable additional datasets for performance evaluation. The implementation and analysis shall focus primarily on the Brute Force approach for the Closest Pair and Convex Hull problems.

Requirements

Use the given set of 10 two-dimensional points as the primary dataset for detailed manual calculations and algorithm verification.

2. Represent the input points in Cartesian coordinate form using the x - and y -coordinates.
3. Apply the Brute Force technique to solve the Closest Pair Problem by examining every possible unique pair of points.
4. Calculate the Euclidean distance between each pair of points using the standard Euclidean distance formula.
5. Determine the minimum Euclidean distance and identify the corresponding closest pair of points.

6. Calculate the total number of pairwise distance comparisons for $n=10$ using the formula:

$$\frac{n(n-1)}{2}$$

7. Apply the Brute Force technique to solve the Convex Hull Problem using the same set of two-dimensional points.

8. Consider every possible pair of points as a candidate edge for the Convex Hull.

9. Apply the orientation or cross-product test to determine the relative position of the remaining points with respect to each candidate edge.

10. Identify the candidate edges that satisfy the convex hull condition and determine the points belonging to the final hull.

11. Arrange the identified convex hull points in a suitable clockwise or counter-clockwise order to form the final convex polygon.

12. Develop clear algorithms and pseudocode for both the Brute Force Closest Pair and Brute Force Convex Hull problems.

13. Implement both algorithms using Python in a suitable development environment such as Google Colab.

14. Provide graphical visualizations of the input points and clearly highlight the closest pair.

15. Provide a graphical representation of the Convex Hull by connecting the identified hull vertices in the correct order.

16. Record the intermediate calculations required to verify the correctness of both algorithms.

17. Analyze the time complexity and space complexity of the Brute Force Closest Pair algorithm.

18. Analyze the time complexity and space complexity of the Brute Force Convex Hull algorithm.

19. Determine the number of candidate edges considered by the Convex Hull Brute Force method for $n=10$.

20. Evaluate the performance of the algorithms using additional datasets with increasing numbers of points.

21. Measure relevant performance parameters such as execution time, number of distance comparisons and number of orientation checks.

22. Present the experimental results using appropriate tables, graphs and performance charts.

23. Compare the theoretical complexity of the algorithms with their experimentally observed performance.

24. Where appropriate, compare the Brute Force methods with more efficient approaches such as Divide-and-Conquer for the Closest Pair problem and Graham Scan or similar efficient methods for the Convex Hull problem.

25. Clearly document the dataset source, data preparation, experimental environment, algorithmic methodology and evaluation procedure.

Constraints

1. The primary problem must use exactly the specified set of 10 two-dimensional points for the detailed manual analysis.
2. Each point must be represented using two numerical coordinates (x,y)(x,y).
3. Duplicate points should be avoided unless a specific experiment is designed to study their effect.
4. For the Closest Pair problem, each unique pair should be considered only once; the same pair should not be evaluated again in reverse order.
5. For $n=10$, exactly 45 unique point pairs are possible:

$$10(10-1)/2 = 45$$

6. For the Convex Hull problem, all possible point pairs may be considered as candidate edges under the Brute Force approach.
7. For $n=10$, the maximum number of candidate edges is also 45:
$$10(10-1)/2 = 45$$
8. The orientation calculation must correctly distinguish between clockwise, counter-clockwise and collinear configurations.
9. Numerical calculations should use sufficient precision to avoid incorrect identification of the minimum distance due to rounding.
10. The same programming environment and hardware conditions should be used when comparing different algorithmic approaches.
11. Experimental execution times should be measured using actual program execution rather than estimated values.
12. Input sizes should be selected according to the computational feasibility of the Brute Force algorithms because their computational cost increases rapidly as n becomes large.
13. The experimental analysis should clearly distinguish between theoretical complexity and actual measured execution time.
14. For very large datasets, the experiment should be limited by available memory, processor capability and reasonable execution time.
15. All graphs and tables should be generated from the actual experimental results obtained from the implementation.
16. The Convex Hull visualization should correctly connect the hull vertices and return to the starting vertex to form a closed polygon.
17. The study should clearly state assumptions related to duplicate points, collinear points, numerical precision and dataset generation.

18. Any external datasets, software libraries, algorithms or reference materials used in the assignment must be properly acknowledged in the References section.

19. The implementation should produce reproducible results by documenting the dataset, input size, execution environment and experimental procedure.

20. The final analysis should identify the advantages and limitations of the Brute Force approach rather than presenting only the final numerical answers.

Expected Experimental Analysis

For performance evaluation, additional point datasets may be generated or obtained from a publicly available coordinate dataset. Different input sizes can be tested, for example:

Input Size	Purpose
10	Detailed manual verification
50	Small-scale experiment
100	Small-scale performance test
500	Medium-scale experiment
1,000	Larger performance test
5,000	Scalability analysis
10,000	Stress testing where computationally feasible

The exact maximum input size should be determined based on the execution capability of the Brute Force implementation. Actual measured results should be reported rather than fabricated values.

The experimental study should record:

Parameter	Closest Pair	Convex Hull
Input size n	✓	✓
Execution time	✓	✓
Number of comparisons	✓	✓
Number of orientation tests	—	✓
Result	✓	✓
Memory usage	✓	✓

D. Student Work

1. Problem Understanding and Formulation

The first stage of the assignment is to understand the Closest Pair and Convex Hull problems and formulate them as computational geometry problems that can be solved using the Brute Force technique.

Problem Analysis

The given dataset consists of 10 two-dimensional points:

$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$
 $P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$

The points are represented in a Cartesian coordinate system, where each point is represented as:

$$P_i=(x_i,y_i) \quad P_{-i}=(x_{-i},y_{-i})$$

The assignment contains two major computational problems.

Closest Pair Problem:

The objective is to identify the two points having the smallest Euclidean distance among all possible pairs of points.

Convex Hull Problem:

The objective is to identify the smallest convex polygon that contains all the given points.

The Brute Force strategy is selected because it provides a direct and systematic way of solving both problems. Instead of eliminating possible solutions using advanced techniques, the algorithm examines all relevant candidates.

For the Closest Pair problem, every unique pair of points is considered.

For $n=10$:

$$\text{Number of pairs} = \frac{10(10-1)}{2} = 45$$

Therefore, 45 unique point pairs must be considered.

For the Convex Hull problem, every pair of points can be considered as a potential hull edge. For 10 points, there are also 45 possible candidate edges.

The problem therefore provides a suitable example for understanding the relationship between **input size, number of operations and algorithmic complexity**.

Brute Force Algorithm/Pseudocode for Both Problems

The Closest Pair Brute Force algorithm follows these basic steps:

- Start with the first point.
- Select another point that has not yet been compared with the selected point.
- Calculate the Euclidean distance between the two points.
- Compare the calculated distance with the current minimum distance.
- Update the minimum distance and closest pair if necessary.
- Continue until every unique pair has been examined.
- Return the closest pair and minimum distance.

The Convex Hull Brute Force algorithm follows a different process:

- Generate every possible pair of points.
- Treat each pair as a candidate hull edge.
- Consider the line formed by the two candidate points.
- Calculate the orientation of every remaining point with respect to the candidate edge.
- Check whether all remaining points lie on the same side of the line.
- If the condition is satisfied, identify the candidate pair as a possible hull edge.
- Collect the points associated with valid hull edges.
- Arrange the hull points in clockwise or counter-clockwise order.
- Construct the final convex polygon.

Detailed algorithms and pseudocode will be presented in the later implementation chapter.

Step-by-Step Calculations

For the Closest Pair problem, the Euclidean distance between two points is calculated using:

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

For example, consider:

$$P_1 = (2, 3)$$

and

$$P_2 = (5, 8)$$

The distance is:

$$\begin{aligned} d(P_1, P_2) &= \sqrt{(5-2)^2 + (8-3)^2} = \sqrt{3^2 + 5^2} \\ &= \sqrt{9+25} = \sqrt{34} \approx 5.831 \end{aligned}$$

Similar calculations are performed for all 45 unique pairs.

For the Convex Hull problem, the orientation of three points is determined using the cross-product expression:

$$\text{Orientation}(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$$

The result determines whether point C lies to the left or right of the directed line AB, or whether the three points are collinear.

These mathematical calculations provide the basis for determining both the closest pair and the convex hull.

Closest-Pair Result

After calculating the distances between all 45 unique pairs, the minimum distance is identified.

For the given dataset:

$$P_3 = (9, 4)$$

and

$$P_5 = (7, 2)$$

form the closest pair.

Their distance is:

$$\begin{aligned} d(P_3, P_5) &= \sqrt{(7-9)^2 + (2-4)^2} = \sqrt{(-2)^2 + (-2)^2} \\ &= \sqrt{4+4} = \sqrt{8} = 2\sqrt{2} \approx 2.828 \end{aligned}$$

Therefore, the closest pair is:

$(9,4)$ and $(7,2)$ \boxed{(9,4)\text{ and }(7,2)}

with minimum Euclidean distance:

2.828 \boxed{2.828}

Convex Hull Result

The Brute Force Convex Hull analysis identifies the outermost boundary points from the given dataset.

The hull points are:

$(2,3), (7,2), (15,6), (10,14), (6,12), (1,7)$ \boxed{(2,3), (7,2), (15,6), (10,14), (6,12), (1,7)}

These points form the boundary of the smallest convex polygon containing all ten input points.

The remaining points lie inside this polygon and therefore do not belong to the outer convex boundary.

The final hull can be represented in counter-clockwise order as:

$(2,3) \rightarrow (7,2) \rightarrow (15,6) \rightarrow (10,14) \rightarrow (6,12) \rightarrow (1,7) \rightarrow (2,3)$
 $(2,3) \rightarrow (7,2) \rightarrow (15,6) \rightarrow (10,14) \rightarrow (6,12) \rightarrow (1,7) \rightarrow (2,3)$

The final edge connects the last hull point back to the first point, producing a closed polygon.

Graphical Representation of the Points and Convex Hull

A coordinate-plane visualization shall be developed using Python and Matplotlib.

The graph should display:

- All 10 input points.
- Point labels P_1 through P_{10} .
- The closest pair highlighted separately.
- The six Convex Hull points.
- Lines connecting the Convex Hull vertices.
- A closed polygon representing the final hull.
- Coordinate axes.
- Appropriate axis labels.
- A legend identifying the closest pair and hull.

The visualization provides an intuitive understanding of the spatial relationship among the points and makes it easier to verify the calculated results.

Time and Space Complexity Analysis

For the Closest Pair Brute Force algorithm, every unique pair is examined.

The number of comparisons is:

$\frac{n(n-1)}{2}$

Therefore:

$$T(n)=O(n^2) \quad T(n)=O(n^2)$$

The space requirement apart from the input storage is constant because only the current minimum distance and corresponding pair need to be maintained.

Therefore:

$$S(n)=O(1) \quad S(n)=O(1)$$

for auxiliary space.

For the Brute Force Convex Hull algorithm, there can be:

$$\frac{n(n-1)}{2}$$

candidate edges.

For each candidate edge, the remaining points may need to be checked. Therefore, the number of operations can grow approximately as:

$$O(n^2) \times O(n) = O(n^3) \quad O(n^2) \times O(n) = O(n^3)$$

Hence, the time complexity is:

$$T(n)=O(n^3) \quad T(n)=O(n^3)$$

The auxiliary space depends on how candidate edges and hull points are stored, but it can generally be maintained within:

$$S(n)=O(n) \quad S(n)=O(n)$$

apart from the input representation.

Comparative Analysis

The two Brute Force algorithms have different computational requirements.

Feature	Closest Pair	Convex Hull
Main operation	Distance calculation	Orientation test
Candidate combinations	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
Basic time complexity	$O(n^2)$	$O(n^3)$
Auxiliary space	$O(1)$	$O(n)$
Main mathematical concept	Euclidean distance	Cross product/orientation
Output	Closest pair	Convex polygon
Scalability	Better	More computationally expensive

The comparison demonstrates that although both algorithms use a Brute Force strategy, their computational costs are different.

The Convex Hull algorithm performs an additional examination of remaining points for each candidate edge, resulting in cubic time complexity.

Final Conclusion

The Brute Force approach successfully solves both computational geometry problems for the given dataset.

For the Closest Pair problem, all 45 possible pairs are examined and the minimum distance is found between:

$(9,4)$ and $(7,2)$

with distance:

$2\sqrt{2} \approx 2.828$

For the Convex Hull problem, the boundary points are identified as:

$(2,3), (7,2), (15,6), (10,14), (6,12), (1,7)$

The study demonstrates that Brute Force algorithms are straightforward to design and implement and provide exact results for small datasets. However, their computational cost increases rapidly as the number of points increases. This makes complexity analysis essential when selecting an algorithm for larger real-world applications.

2. Application of Course Knowledge

This assignment applies several concepts from **Design and Analysis of Algorithms** and demonstrates how theoretical knowledge can be used to solve practical computational geometry problems.

Principles

The primary algorithmic principle applied is the **Brute Force strategy**.

The algorithm systematically examines possible solutions rather than attempting to reduce the search space through advanced optimization.

Other principles applied include:

- Problem decomposition.
- Exhaustive search.
- Iterative comparison.
- Optimization through minimum selection.
- Geometric testing.
- Algorithm correctness.
- Complexity analysis.
- Performance evaluation.

Theories

The following theoretical concepts are applied:

- Algorithm design paradigms.
- Computational geometry.
- Asymptotic analysis.
- Big-O notation.

- Best-case analysis.
- Average-case analysis.
- Worst-case analysis.
- Search-space analysis.
- Algorithm scalability.

The assignment demonstrates how theoretical complexity relates to actual execution behaviour.

Mathematical Models

Several mathematical models are required.

The Euclidean distance model is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The number of unique pairs is:

$$C(n, 2) = \frac{n(n-1)}{2}$$

The orientation model is:

$$O(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$$

These mathematical models transform geometric concepts into computational operations.

Algorithms

The assignment applies:

- Brute Force Closest Pair algorithm.
- Brute Force Convex Hull algorithm.
- Euclidean distance calculation.
- Cross-product orientation test.
- Candidate-edge examination.
- Point-ordering procedure.

Efficient algorithms may also be considered for comparison, including:

- Divide-and-Conquer Closest Pair.
- Graham Scan.
- Jarvis March.

Engineering Concepts

The assignment demonstrates several engineering concepts, including:

- Systematic problem solving.
- Computational efficiency.
- Performance measurement.
- Resource utilization.
- Scalability.
- Experimental validation.

- Design trade-offs.

An algorithm is not evaluated only on whether it produces a correct answer. Its computational cost and suitability for different input sizes must also be considered.

Scientific Concepts

The study involves:

- Coordinate geometry.
- Euclidean distance.
- Vector direction.
- Cross products.
- Orientation.
- Convexity.
- Spatial relationships.

These mathematical and scientific concepts are converted into computational procedures.

Design Methodologies

The solution follows a structured algorithm design methodology:

1. Understand the problem.
2. Identify inputs and outputs.
3. Develop the mathematical formulation.
4. Select the Brute Force strategy.
5. Develop the algorithm.
6. Write pseudocode.
7. Implement the algorithm.
8. Test the implementation.
9. Measure performance.
10. Analyze the results.
11. Compare alternative approaches.
12. Draw conclusions.

Necessary Calculations and Reasoning

The assignment requires calculations such as:

$$10(10-1)/2 = 45$$

This establishes the number of unique point pairs.

For the closest pair:

$$d((9,4),(7,2)) = \sqrt{(7-9)^2 + (2-4)^2} = \sqrt{8} = 2.828$$

The mathematical reasoning is directly connected to the algorithmic implementation.

3. Solution / Design / Methodology

3.1 Proposed Solution

The proposed solution consists of two independent but related Brute Force computational geometry algorithms.

The first algorithm solves the Closest Pair problem.

The second algorithm solves the Convex Hull problem.

Both algorithms use the same input dataset so that their behaviour can be analyzed under identical conditions.

The overall methodology is:

Input Points → Mathematical Formulation → Brute Force Algorithm → Python Implementation → Results → Visualization → Complexity Analysis → Experimental Evaluation

3.2 Closest Pair Methodology

The Closest Pair algorithm begins by accepting a collection of two-dimensional points.

Every unique pair is generated using two nested loops.

For each pair:

1. Extract the coordinates.
2. Calculate the difference in xx-coordinates.
3. Calculate the difference in yy-coordinates.
4. Square both differences.
5. Add the squared differences.
6. Calculate the square root.
7. Compare the distance with the current minimum.
8. Update the minimum when a smaller distance is found.

After all 45 pairs have been examined, the algorithm returns the closest pair.

3.3 Convex Hull Methodology

The Convex Hull methodology uses candidate-edge testing.

For every pair of points AA and BB:

1. Treat ABAB as a candidate edge.
2. Select every other point CC.
3. Calculate $\text{Orientation}(A,B,C)$.
4. Determine the sign of the orientation.
5. Check whether the points are consistently on one side of the candidate line.
6. If the candidate satisfies the hull condition, store the edge.
7. Extract the points participating in valid hull edges.
8. Arrange the points around their geometric boundary.
9. Connect the ordered points.
10. Close the polygon by connecting the final point to the first point.

This provides an exhaustive method for determining the convex boundary.

3.4 Alternative Solution – Divide-and-Conquer Closest Pair

A more efficient solution to the Closest Pair problem is the Divide-and-Conquer method.

Instead of comparing every pair, the point set is divided into smaller subsets.

The basic approach is:

1. Sort points according to their xx-coordinate.
2. Divide the points into two halves.
3. Recursively find the closest pair in each half.
4. Determine the minimum of the two distances.
5. Examine points near the dividing line.
6. Check whether a closer cross-boundary pair exists.
7. Return the overall minimum.

The expected time complexity is:

$$O(n \log n)$$

which is substantially better than the Brute Force:

$$O(n^2)$$

for sufficiently large datasets.

3.5 Alternative Solution – Efficient Convex Hull Algorithm

For Convex Hull, algorithms such as Graham Scan can be considered as alternatives.

Graham Scan generally requires:

$$O(n \log n)$$

time due primarily to sorting.

This is significantly better than the Brute Force Convex Hull approach:

$$O(n^3)$$

The comparison allows the assignment to demonstrate why algorithm selection is important when processing large datasets.

3.6 Experimental Methodology

The implementation will be tested using different numbers of input points.

Possible input sizes include:

10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000

subject to computational feasibility.

For each input size, the following measurements will be collected:

- Number of distance comparisons.
- Number of orientation tests.
- Execution time.
- Number of hull vertices.
- Minimum distance.
- Memory usage where measurable.

The same computer environment and Python environment should be used for all experiments.

3.7 Visualization Methodology

Python and Matplotlib will be used to generate graphical outputs.

The visualization will include:

1. Original distribution of points.
2. Point labels.
3. Closest pair.
4. Convex Hull boundary.
5. Hull vertices.
6. Execution-time graphs.
7. Comparison graphs.

These visualizations provide evidence that the algorithm has been implemented correctly.

3.8 Validation Methodology

The implementation will be validated by comparing the program output with manually calculated results.

For the primary dataset:

Closest Pair:

(9,4),(7,2)(9,4),(7,2)

Minimum Distance:

$8 \approx 2.828 \sqrt{8} \approx 2.828$

Convex Hull:

(2,3),(7,2),(15,6),(10,14),(6,12),(1,7)(2,3),(7,2),(15,6),(10,14),(6,12),(1,7)

The experimental results will then be analyzed to determine whether the observed increase in execution time is consistent with the theoretical complexity.

3.9 Final Methodology Flow

The complete methodology of the assignment can therefore be summarized as:

Step 1: Define the computational geometry problems.

Step 2: Prepare the 10-point dataset.

Step 3: Formulate the Euclidean distance equation.

Step 4: Calculate all unique point-pair distances.

Step 5: Identify the closest pair.

Step 6: Formulate the orientation equation.

Step 7: Generate candidate Convex Hull edges.

Step 8: Perform orientation tests.

Step 9: Identify valid hull edges.

Step 10: Construct the final Convex Hull.

Step 11: Implement both algorithms in Python.

Step 12: Generate graphical visualizations.

Step 13: Test increasing input sizes.

Step 14: Record execution time and operation counts.

Step 15: Analyze time and space complexity.

Step 16: Compare Brute Force with efficient alternatives.

Step 17: Evaluate advantages, limitations and practical applications.

Step 18: Present the final conclusions and recommendations.

Sure. I'll remove the horizontal separator lines and keep the content clean for direct pasting into Word.

4. Use of Modern Tools

Modern computational tools are used to design, implement, test, visualize and evaluate the Brute Force algorithms for the Closest Pair and Convex Hull problems. These tools help convert the mathematical formulation into executable programs and provide experimental evidence for the theoretical analysis.

4.1 Python

Python is used as the primary programming language for implementing both algorithms. Python provides simple syntax and useful libraries for numerical calculations, data processing, algorithm implementation and visualization.

The following tasks are performed using Python:

- Representing two-dimensional coordinate points.
- Calculating Euclidean distances.
- Performing pairwise comparisons.
- Implementing the Brute Force Closest Pair algorithm.
- Calculating orientation using the cross-product formula.
- Implementing the Brute Force Convex Hull algorithm.
- Measuring algorithm execution time.
- Counting distance comparisons and orientation tests.
- Generating graphical outputs.
- Conducting experiments using different input sizes.

4.2 Google Colab

Google Colab can be used as the execution environment for the Python implementation. It provides a browser-based programming environment where Python programs can be written and executed without requiring complex local configuration.

Google Colab is useful for:

- Writing and executing Python source code.
- Testing different datasets.
- Measuring execution time.
- Generating graphs.
- Displaying Closest Pair and Convex Hull visualizations.

- Saving experimental outputs.
- Reproducing the algorithmic experiments.

Screenshots of the executed code and generated outputs can be included in the assignment as evidence of tool usage.

4.3 NumPy

NumPy can be used for numerical operations and efficient handling of coordinate data.

It can assist with:

- Storing coordinate values.
- Performing numerical calculations.
- Generating larger experimental datasets.
- Managing arrays of points.
- Supporting mathematical operations.

The main Brute Force algorithm should still be implemented explicitly so that the number of comparisons and operations can be measured clearly.

4.4 Matplotlib

Matplotlib is used for graphical visualization of the results.

The following visualizations can be generated:

- Scatter plot of all input points.
- Closest Pair visualization.
- Convex Hull visualization.
- Execution time versus input size.
- Number of operations versus input size.
- Comparison of different algorithms.

The graphical outputs make it easier to verify the correctness of the geometric calculations.

4.5 Python Timing Functions

Python timing functions can be used to measure the execution time of the algorithms.

The experiment should record:

- Start time.

- Algorithm execution.
- End time.
- Total execution duration.

Execution time should be recorded for each tested input size.

4.6 Git and Version Control

Git can be used to maintain different versions of the source code.

Version control can help in:

- Tracking changes.
- Maintaining previous versions.
- Organizing source code.
- Recovering earlier implementations.
- Supporting reproducibility.
- Managing project files.

4.7 Evidence of Tool Usage

The assignment should provide appropriate evidence of the tools used.

The evidence may include:

- Screenshot of Python/Google Colab source code.
- Screenshot of program execution.
- Screenshot of Closest Pair output.
- Screenshot of Convex Hull output.
- Screenshot of generated graphs.
- Experimental results tables.
- Dataset used for testing.
- Source code used for implementation.

5. Results and Validation

The Results and Validation section presents the outcomes obtained from the implementation of the Brute Force Closest Pair and Convex Hull algorithms. Both correctness and computational performance should be evaluated.

5.1 Closest Pair Result

For the given set of 10 points, the algorithm examines every unique pair.

The total number of unique pairs is:

$$10(10-1)/2 = 45$$

The minimum distance is obtained between:

(9,4) and (9,4)

and

(7,2) and (7,2)

The distance is:

$$d = \sqrt{(9-7)^2 + (4-2)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.828$$

Therefore:

Closest Pair = (9,4) and (7,2)

Minimum Distance = 2.828 units

The result can be validated by confirming that none of the remaining 44 pairs has a smaller Euclidean distance.

5.2 Convex Hull Result

The Brute Force Convex Hull algorithm examines possible candidate edges and applies the orientation test.

For 10 points, the number of possible point pairs is:

$$10(10-1)/2 = 45$$

The resulting Convex Hull points are:

(2,3), (7,2), (15,6), (10,14), (6,12), (1,7), (2,3), (7,2), (15,6), (10,14), (6,12), (1,7)

These points form the outer boundary of the smallest convex polygon containing all the given points.

The remaining points lie inside the polygon and therefore are not included as Convex Hull vertices.

5.3 Primary Results Table

Parameter	Result
Number of input points	10
Number of unique pairs	45
Closest Pair	(9,4) and (7,2)
Minimum distance	2.828 units
Candidate point pairs for Convex Hull	45
Number of Convex Hull vertices	6
Algorithm	Brute Force

5.4 Experimental Performance Results

Additional experiments should be conducted using increasing numbers of points.

Possible input sizes include:

Input Size	Distance Comparisons	Closest Pair Time	Orientation Tests	Convex Hull Time
10	45	Actual result	Actual result	Actual result
50	1,225	Actual result	Actual result	Actual result
100	4,950	Actual result	Actual result	Actual result
500	124,750	Actual result	Actual result	Actual result
1,000	499,500	Actual result	Actual result	Actual result
5,000	12,497,500	Actual result	Actual result	Actual result

The execution-time values should be obtained directly from the Python implementation. Estimated or fabricated execution times should not be used.

5.5 Graphical Results

The following graphs should be included in the final report:

Graph 1: Input Points

A scatter plot showing all ten input points with appropriate point labels.

Graph 2: Closest Pair

A graph showing all input points with the closest pair clearly highlighted.

Graph 3: Convex Hull

A coordinate graph showing all points and the resulting Convex Hull polygon.

Graph 4: Execution Time versus Input Size

A graph showing how execution time changes as the number of input points increases.

Graph 5: Number of Operations versus Input Size

A graph showing the increase in distance comparisons and orientation tests.

Graph 6: Algorithm Comparison

A graph comparing Brute Force performance with more efficient algorithms where such algorithms are implemented.

5.6 Validation of Closest Pair

The Closest Pair implementation can be validated using the following procedure:

1. Generate all unique pairs of points.
2. Calculate the Euclidean distance for every pair.
3. Store the smallest distance found.
4. Store the corresponding pair of points.
5. Compare the program result with the manually calculated result.
6. Verify that 45 comparisons are performed for $n=10$.
7. Confirm that the minimum distance is approximately 2.828.

If all these conditions are satisfied, the implementation is considered correct for the given dataset.

5.7 Validation of Convex Hull

The Convex Hull implementation can be validated by:

- Checking all possible candidate point pairs.
- Applying the orientation test.
- Identifying valid hull edges.
- Confirming that the selected points lie on the outer boundary.
- Confirming that the remaining points lie inside or on the boundary.
- Ensuring that the final hull forms a closed polygon.
- Comparing the program output with the manually determined Convex Hull.

5.8 Overall Validation

The proposed solution satisfies the assignment requirements when:

- Both algorithms produce correct results.
- All required point pairs are examined.
- Candidate Convex Hull edges are evaluated.
- Mathematical calculations agree with program output.
- Complexity analysis agrees with the algorithmic structure.

- Graphical outputs correctly represent the results.
- Experimental measurements are recorded.
- Results can be reproduced using the documented implementation.

6. Analysis and Engineering Decision

The theoretical and experimental results are analyzed to determine the suitability of the Brute Force approach for different input sizes.

6.1 Comparison of Alternative Algorithms

The Brute Force Closest Pair algorithm can be compared with the Divide-and-Conquer approach.

Feature	Brute Force	Divide-and-Conquer
Basic approach	Examine every pair	Divide points into subsets
Time Complexity	$O(n^2)$	$O(n \log n)$
Implementation	Simple	More complex
Small datasets	Suitable	Suitable
Large datasets	Less efficient	More efficient
Main advantage	Simplicity	Scalability

For Convex Hull, the Brute Force method can be compared with Graham Scan.

Feature	Brute Force	Graham Scan
Basic approach	Examine candidate edges	Sort and scan points
Time Complexity	$O(n^3)$	$O(n \log n)$
Implementation	Simple concept	More sophisticated
Small datasets	Suitable	Suitable
Large datasets	Computationally expensive	More efficient
Main advantage	Easy verification	Better scalability

6.2 Advantages of Brute Force

The major advantages are:

- Simple to understand.
- Easy to implement.
- Directly follows the problem definition.
- Easy to debug.
- Provides exact results.
- Useful for verifying advanced algorithms.
- Suitable for small datasets.
- Provides a clear demonstration of algorithmic complexity.

6.3 Limitations of Brute Force

The major limitations are:

- Large number of comparisons.
- High computational cost for large datasets.
- Poor scalability.
- Repeated examination of possible solutions.
- Increasing execution time as n increases.
- Convex Hull computation becomes particularly expensive because of its higher-order complexity.

6.4 Trade-Off Analysis

The main trade-off is between **simplicity and computational efficiency**.

The Brute Force method examines all possible candidates. This makes the algorithm straightforward but increases the number of operations.

More efficient algorithms reduce the search space but generally require more complicated implementation.

For example:

$$O(n \log n) < O(n^2) \quad O(n \log n) < O(n^2)$$

for sufficiently large values of n .

Similarly:

$$O(n \log n) < O(n^3) \quad O(n \log n) < O(n^3)$$

Therefore, algorithm selection should depend on the size of the input and the computational requirements of the application.

6.5 Engineering Decision

For the given dataset of 10 points, the Brute Force approach is selected because:

- The dataset is small.
- All possible solutions can be examined.
- The calculations can be manually verified.
- The implementation is easy to understand.
- The algorithm demonstrates the principles of exhaustive search.

For large datasets, more efficient algorithms should be selected to reduce execution time and computational resource requirements.

Therefore, the Brute Force approach is appropriate as the **primary educational and analytical solution**, while efficient algorithms can be used for comparison and scalability analysis.

7. Broader Considerations

7.1 Sustainability

Algorithmic efficiency can contribute to sustainable computing.

Algorithms requiring fewer operations may reduce:

- CPU usage.
- Processing time.
- Electricity consumption.
- Computing infrastructure requirements.

Although the difference may be small for ten points, it becomes important when processing millions of points.

7.2 Environment

Large-scale computing systems consume electrical energy. Efficient algorithms can reduce unnecessary computational processing.

The study demonstrates that algorithm selection can influence the amount of computational resources required to solve a problem.

7.3 Society

Computational geometry algorithms have applications in:

- Geographic Information Systems.
- Digital mapping.
- Navigation.
- Urban planning.
- Robotics.
- Computer graphics.
- Image processing.
- Transportation systems.
- Location-based services.
- Spatial data analysis.

These applications demonstrate the practical importance of computational geometry.

7.4 Safety

Geometric algorithms can be used in systems involving spatial decision-making.

Examples include:

- Robot navigation.
- Collision detection.
- Autonomous vehicles.
- Aircraft path planning.
- Vehicle path planning.
- Geographic safety analysis.

Correct implementation is essential because errors in geometric calculations can lead to incorrect spatial decisions.

7.5 Ethics

When computational geometry algorithms are applied to real-world location or spatial datasets, ethical considerations may arise.

Important considerations include:

- Data privacy.
- Responsible data collection.
- Data accuracy.
- Appropriate use of location information.
- Transparency of computational decisions.

7.6 Economics

Algorithm efficiency can affect the cost of computational resources.

For large applications, inefficient algorithms can require:

- More processing power.
- Longer execution times.
- Additional server resources.
- Higher operational costs.

Efficient algorithm selection can therefore provide economic benefits.

7.7 Accessibility

Python and Google Colab make algorithm development accessible to students and researchers without requiring expensive specialized software.

Graphical visualization also makes computational geometry concepts easier to understand.

7.8 Professional Responsibility

A software professional should:

- Verify algorithm correctness.
- Test programs thoroughly.
- Document assumptions.
- Report experimental results honestly.
- Avoid fabricating performance measurements.
- Properly cite external sources.
- Clearly state limitations.
- Select algorithms according to application requirements.

These practices contribute to the development of reliable and responsible computational systems.

8. Conclusion

8.1 Proposed Solution

The assignment implemented and analyzed Brute Force algorithms for two important computational geometry problems: the Closest Pair Problem and the Convex Hull Problem.

The same set of 10 two-dimensional points was used for both problems:

$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$

The Closest Pair algorithm examined every unique pair and calculated the Euclidean distance.

The Convex Hull algorithm examined candidate edges and used orientation testing to identify the points forming the outer boundary.

8.2 Major Findings

For $n=10$, the number of unique point pairs is:

$$\frac{10(10-1)}{2} = 45$$

The closest pair is:

$(9,4)$ and $(7,2)$

The minimum distance is:

$$8 \approx 2.828 \sqrt{8} \approx 2.828$$

The Convex Hull consists of:

$$(2,3),(7,2),(15,6),(10,14),(6,12),(1,7)$$

The study demonstrates that the Brute Force Closest Pair algorithm has a time complexity of:

$$O(n^2)$$

The Brute Force Convex Hull approach has approximately:

$$O(n^3)$$

time complexity when every candidate edge is checked against the remaining points.

8.3 Achievement of Requirements

The assignment achieves the major requirements by:

- Formulating both computational geometry problems.
- Applying the Brute Force technique.
- Performing mathematical calculations.
- Developing algorithms and pseudocode.
- Implementing the algorithms using Python.
- Determining the Closest Pair.
- Determining the Convex Hull.
- Creating graphical representations.
- Performing time and space complexity analysis.
- Conducting experimental evaluation.
- Comparing alternative algorithms.
- Discussing advantages and limitations.

8.4 Limitations

The main limitation of the Brute Force approach is its computational cost for large datasets.

The Closest Pair algorithm requires:

$$O(n^2)$$

time.

The Brute Force Convex Hull approach can require:

$O(n^3)$

time.

Therefore, these methods are more suitable for small datasets, educational demonstrations and verification of other algorithms than for very large-scale applications.

8.5 Possible Improvements

Future work can include:

- Implementing Divide-and-Conquer Closest Pair.
- Implementing Graham Scan.
- Implementing Jarvis March.
- Testing larger datasets.
- Measuring memory consumption.
- Comparing CPU usage.
- Performing repeated experiments.
- Improving the Python implementation.
- Developing interactive visualization.
- Investigating spatial indexing techniques.

Overall, the assignment demonstrates the importance of considering both **correctness and computational efficiency** when designing algorithms.

9. Student Reflection

9.1 Learning Beyond Classroom Teaching

This assignment provided practical knowledge beyond the theoretical concepts discussed in the classroom.

I learned that an algorithm can produce the correct result but may not always be suitable for large datasets. The Brute Force approach helped me understand how examining every possible solution affects computational performance.

I gained practical experience in converting mathematical formulas into executable Python programs. Implementing the Euclidean distance calculation helped me understand how mathematical concepts can be directly translated into programming operations.

The Convex Hull problem provided a deeper understanding of coordinate geometry, cross products and orientation tests. I learned how the relative positions of points can be determined computationally using mathematical expressions.

Another important learning outcome was understanding the relationship between theoretical complexity and actual execution time. As the number of input points increases, the number of operations also increases, which affects execution time.

I also learned the importance of visualization. Plotting the points and Convex Hull provides an intuitive way to verify whether the algorithm has produced the expected result.

The assignment improved my ability to:

- Analyze computational problems.
- Develop algorithms.
- Write pseudocode.
- Implement Python programs.
- Debug programs.
- Test different input sizes.
- Measure execution time.
- Interpret experimental results.
- Generate graphical outputs.
- Compare algorithmic approaches.
- Document technical work.

9.2 Improvements with Additional Time and Resources

If additional time and resources were available, I would implement more efficient algorithms and compare their performance with the Brute Force approach.

For the Closest Pair problem, I would implement the Divide-and-Conquer method and compare its $O(n \log n)$ performance with the $O(n^2)$ Brute Force approach.

For the Convex Hull problem, I would implement Graham Scan and Jarvis March and compare their performance with the Brute Force method.

I would also use larger datasets to study scalability more effectively.

Additional performance parameters could include:

- CPU utilization.
- Memory usage.
- Number of mathematical operations.
- Average execution time.

- Best-case execution time.
- Worst-case execution time.
- Dataset-dependent performance.

An interactive graphical interface could also be developed so that users can generate points, run the algorithms and observe the Closest Pair and Convex Hull dynamically.

These improvements would provide a more comprehensive understanding of algorithm performance and improve the practical value of the project.

10. References

The final report should use a consistent citation style such as IEEE format. References should acknowledge the textbooks, research papers, software libraries, datasets and other resources actually used during the assignment.

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Pearson, 2006.
- [3] M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin, Germany: Springer, 2008.
- [4] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. New York, NY, USA: Springer, 1985.
- [5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [6] J. O'Rourke, *Computational Geometry in C*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 1998.
- [7] Python Software Foundation, *Python Documentation*, Python Software Foundation.
- [8] NumPy Developers, *NumPy Documentation*, NumPy Project.
- [9] Matplotlib Development Team, *Matplotlib Documentation*, Matplotlib Project.
- [10] Google, *Google Colaboratory Documentation*, Google Research.

E. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.

Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
		decision with evidence.			
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

PYTHON CODE

```
# =====  
# ANALYTICAL STUDY AND IMPLEMENTATION OF BRUTE FORCE ALGORITHMS  
# FOR CLOSEST PAIR AND CONVEX HULL PROBLEMS  
# =====  
  
import math  
import time  
import random  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# =====  
# 1. GIVEN INPUT DATASET  
# =====  
  
points = [  
    (2, 3),  
    (5, 8),  
    (9, 4),  
    (12, 10),  
    (7, 2),  
    (3, 11),  
    (15, 6),  
    (10, 14),  
    (6, 12),  
    (1, 7)  
]  
  
print("Given Points:")  
for i, point in enumerate(points, start=1):  
    print(f"P {i} = {point}")  
  
# =====
```

2. BRUTE FORCE CLOSEST PAIR ALGORITHM

=====

```
def euclidean_distance(p1, p2):
```

```
    """
```

```
    Calculate Euclidean distance between two points.
```

```
    """
```

```
    return math.sqrt(
```

```
        (p1[0] - p2[0]) ** 2 +
```

```
        (p1[1] - p2[1]) ** 2
```

```
    )
```

```
def brute_force_closest_pair(points):
```

```
    """
```

```
    Brute Force Closest Pair Algorithm.
```

```
    Checks every unique pair of points and returns:
```

```
    - Closest pair
```

```
    - Minimum distance
```

```
    - Number of distance comparisons
```

```
    """
```

```
    n = len(points)
```

```
    if n < 2:
```

```
        return None, float('inf'), 0
```

```
    min_distance = float('inf')
```

```
    closest_pair = None
```

```
    comparisons = 0
```

```
    for i in range(n):
```

```
        for j in range(i + 1, n):
```

```
            comparisons += 1
```

```

        distance = euclidean_distance(
            points[i],
            points[j]
        )

        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

    return closest_pair, min_distance, comparisons

# =====
# 3. FIND CLOSEST PAIR
# =====

closest_pair, minimum_distance, distance_comparisons = \
    brute_force_closest_pair(points)

print("\n" + "=" * 60)
print("CLOSEST PAIR RESULT")
print("=" * 60)

print("Closest Pair:", closest_pair)
print("Minimum Distance:", minimum_distance)
print("Distance Comparisons:", distance_comparisons)

expected_comparisons = len(points) * (len(points) - 1) // 2

print("Expected Comparisons:",
      expected_comparisons)

# =====
# 4. DISPLAY ALL PAIRWISE DISTANCES

```

```
# =====
```

```
print("\n" + "=" * 60)
print("PAIRWISE DISTANCE CALCULATIONS")
print("=" * 60)
```

```
pair_number = 1
```

```
for i in range(len(points)):
    for j in range(i + 1, len(points)):
```

```
        p1 = points[i]
```

```
        p2 = points[j]
```

```
        distance = euclidean_distance(p1, p2)
```

```
        print(
            f" {pair_number:2d}. "
            f" {p1} - {p2} = {distance:.4f} "
        )
```

```
        pair_number += 1
```

```
# =====
```

```
# 5. ORIENTATION FUNCTION
```

```
# =====
```

```
orientation_count = 0
```

```
def orientation(p, q, r):
```

```
    """
```

```
    Calculate orientation of three points.
```

```
    Returns:
```

0 -> Collinear

1 -> Clockwise

2 -> Counter-clockwise

"""

global orientation_count

orientation_count += 1

```
value = (
    (q[0] - p[0]) * (r[1] - p[1])
    -
    (q[1] - p[1]) * (r[0] - p[0])
)
```

if value == 0:

return 0

if value > 0:

return 2

return 1

```
# =====
# 6. BRUTE FORCE CONVEX HULL
# =====
```

def brute_force_convex_hull(points):

global orientation_count

orientation_count = 0

n = len(points)

```

if n <= 2:
    return points.copy(), 0

hull_edges = set()

# -----
# Examine every possible pair as a candidate edge
# -----

for i in range(n):
    for j in range(i + 1, n):

        left_side = False
        right_side = False

        valid_edge = True

        for k in range(n):

            if k == i or k == j:
                continue

            value = (
                (points[j][0] - points[i][0]) *
                (points[k][1] - points[i][1])
                -
                (points[j][1] - points[i][1]) *
                (points[k][0] - points[i][0])
            )

            if value > 0:
                left_side = True

            elif value < 0:
                right_side = True

```

```

        # If points exist on both sides,
        # the edge is not a hull edge.
        if left_side and right_side:
            valid_edge = False
            break

    if valid_edge:
        hull_edges.add((i, j))

# -----
# Extract hull vertices
# -----

hull_indices = set()

for i, j in hull_edges:
    hull_indices.add(i)
    hull_indices.add(j)

hull_points = [
    points[i]
    for i in hull_indices
]

# -----
# Calculate centroid
# -----

if len(hull_points) > 2:

    center_x = sum(p[0] for p in hull_points) / len(hull_points)
    center_y = sum(p[1] for p in hull_points) / len(hull_points)

    # Sort around centroid
    hull_points.sort(
        key=lambda p: math.atan2(

```

```

        p[1] - center_y,
        p[0] - center_x
    )
)

return hull_points, orientation_count

# =====
# 7. FIND CONVEX HULL
# =====

hull_points, orientation_tests = \
    brute_force_convex_hull(points)

print("\n" + "=" * 60)
print("CONVEX HULL RESULT")
print("=" * 60)

print("Convex Hull Points:")

for i, point in enumerate(hull_points, start=1):
    print(f"H{i} = {point}")

print("\nNumber of Convex Hull Vertices:",
      len(hull_points))

print("Orientation Tests:",
      orientation_tests)

candidate_edges = len(points) * (len(points) - 1) // 2

print("Candidate Edges:",
      candidate_edges)

```

```

# =====
# 8. VISUALIZATION OF INPUT POINTS
# =====

plt.figure(figsize=(10, 7))

x_values = [p[0] for p in points]
y_values = [p[1] for p in points]

plt.scatter(
    x_values,
    y_values,
    s=80,
    label="Input Points"
)

for i, (x, y) in enumerate(points, start=1):
    plt.annotate(
        f"P{i}",
        (x, y),
        xytext=(5, 5),
        textcoords="offset points"
    )

plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title("Given Set of 10 Two-Dimensional Points")
plt.grid(True)
plt.legend()
plt.show()

# =====
# 9. CLOSEST PAIR VISUALIZATION
# =====

```

```
plt.figure(figsize=(10, 7))
```

```
plt.scatter(  
    x_values,  
    y_values,  
    s=80,  
    label="Input Points"  
)
```

```
for i, (x, y) in enumerate(points, start=1):
```

```
    plt.annotate(  
        f"P{i}",  
        (x, y),  
        xytext=(5, 5),  
        textcoords="offset points"  
    )
```

```
cp_x = [  
    closest_pair[0][0],  
    closest_pair[1][0]  
]
```

```
cp_y = [  
    closest_pair[0][1],  
    closest_pair[1][1]  
]
```

```
plt.plot(  
    cp_x,  
    cp_y,  
    linewidth=3,  
    label="Closest Pair"  
)
```

```
plt.scatter(  
    cp_x,
```

```

        cp_y,
        s=150,
        label="Closest Points"
    )

plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title(
    f"Closest Pair "
    f"(Distance = {minimum_distance:.3f})"
)

plt.grid(True)
plt.legend()
plt.show()

```

```

# =====
# 10. CONVEX HULL VISUALIZATION
# =====

```

```

plt.figure(figsize=(10, 7))

plt.scatter(
    x_values,
    y_values,
    s=80,
    label="Input Points"
)

for i, (x, y) in enumerate(points, start=1):
    plt.annotate(
        f"P {i}",
        (x, y),
        xytext=(5, 5),
        textcoords="offset points"
    )

```

```
)

# Close the polygon
closed_hull = hull_points + [hull_points[0]]
```

```
hull_x = [p[0] for p in closed_hull]
hull_y = [p[1] for p in closed_hull]
```

```
plt.plot(
    hull_x,
    hull_y,
    linewidth=3,
    label="Convex Hull"
)
```

```
plt.scatter(
    [p[0] for p in hull_points],
    [p[1] for p in hull_points],
    s=150,
    label="Hull Vertices"
)
```

```
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title("Brute Force Convex Hull")
plt.grid(True)
plt.legend()
plt.show()
```

```
# =====
# 11. PRINT FINAL RESULTS
# =====
```

```
print("\n" + "=" * 60)
print("FINAL RESULTS")
```

```
print("=" * 60)
```

```
print("\nClosest Pair:")
```

```
print(closest_pair)
```

```
print("\nMinimum Euclidean Distance:")
```

```
print(f" {minimum_distance:.4f} ")
```

```
print("\nDistance Comparisons:")
```

```
print(distance_comparisons)
```

```
print("\nConvex Hull:")
```

```
print(hull_points)
```

```
print("\nNumber of Hull Vertices:")
```

```
print(len(hull_points))
```

```
print("\nCandidate Edges:")
```

```
print(candidate_edges)
```

```
print("\nOrientation Tests:")
```

```
print(orientation_tests)
```

```
# =====
```

```
# 12. PERFORMANCE EXPERIMENT
```

```
# =====
```

```
def generate_points(n, seed=42):
```

```
    """
```

```
    Generate n random 2D points.
```

```
    """
```

```
    random.seed(seed)
```

```
    return [
```

```
(
    random.randint(0, 100000),
    random.randint(0, 100000)
)
for _ in range(n)
]
```

```
# Smaller sizes are used because the Brute Force
# Convex Hull algorithm has  $O(n^3)$  complexity.
```

```
input_sizes = [
    10,
    50,
    100,
    200,
    500,
    1000
]
```

```
performance_results = []
```

```
for n in input_sizes:
```

```
    test_points = generate_points(n, seed=42)
```

```
    # -----
    # Closest Pair timing
    # -----
```

```
    start_time = time.perf_counter()
```

```
    _, _, cp_comparisons = \
        brute_force_closest_pair(test_points)
```

```

cp_time = time.perf_counter() - start_time

# -----
# Convex Hull timing
# -----

start_time = time.perf_counter()

_, ch_orientation_tests = \
    brute_force_convex_hull(test_points)

ch_time = time.perf_counter() - start_time

performance_results.append({
    "Input Size": n,
    "Closest Pair Comparisons": cp_comparisons,
    "Closest Pair Time (sec)": cp_time,
    "Convex Hull Orientation Tests": ch_orientation_tests,
    "Convex Hull Time (sec)": ch_time
})

# =====
# 13. PERFORMANCE TABLE
# =====

results_df = pd.DataFrame(
    performance_results
)

print("\n" + "=" * 60)
print("PERFORMANCE RESULTS")
print("=" * 60)

display(results_df)

```

```
# =====  
# 14. EXECUTION TIME GRAPH  
# =====
```

```
plt.figure(figsize=(10, 7))
```

```
plt.plot(  
    results_df["Input Size"],  
    results_df["Closest Pair Time (sec)"],  
    marker="o",  
    linewidth=2,  
    label="Closest Pair Brute Force"  
)
```

```
plt.plot(  
    results_df["Input Size"],  
    results_df["Convex Hull Time (sec)"],  
    marker="s",  
    linewidth=2,  
    label="Convex Hull Brute Force"  
)
```

```
plt.xlabel("Number of Input Points")  
plt.ylabel("Execution Time (seconds)")  
plt.title("Execution Time vs Input Size")  
plt.grid(True)  
plt.legend()  
plt.show()
```

```
# =====  
# 15. CLOSEST PAIR COMPARISON GRAPH  
# =====
```

```
plt.figure(figsize=(10, 7))
```

```
plt.plot(
    results_df["Input Size"],
    results_df["Closest Pair Comparisons"],
    marker="o",
    linewidth=2
)

plt.xlabel("Number of Input Points")
plt.ylabel("Number of Distance Comparisons")
plt.title(
    "Closest Pair: Number of Comparisons vs Input Size"
)

plt.grid(True)
plt.show()
```

```
# =====
# 16. CONVEX HULL ORIENTATION TEST GRAPH
# =====
```

```
plt.figure(figsize=(10, 7))

plt.plot(
    results_df["Input Size"],
    results_df["Convex Hull Orientation Tests"],
    marker="o",
    linewidth=2
)

plt.xlabel("Number of Input Points")
plt.ylabel("Number of Orientation Tests")
plt.title(
    "Convex Hull: Orientation Tests vs Input Size"
)
```

```
plt.grid(True)
```

```
plt.show()
```

```
# =====
```

```
# 17. THEORETICAL NUMBER OF PAIR COMPARISONS
```

```
# =====
```

```
print("\n" + "=" * 60)
```

```
print("THEORETICAL PAIRWISE COMPARISONS")
```

```
print("=" * 60)
```

```
for n in input_sizes:
```

```
    comparisons = n * (n - 1) // 2
```

```
    print(
```

```
        f'n = {n:4d} -> "
```

```
        f'{comparisons:,} comparisons"
```

```
    )
```

```
# =====
```

```
# 18. COMPLEXITY SUMMARY
```

```
# =====
```

```
print("\n" + "=" * 60)
```

```
print("COMPLEXITY ANALYSIS")
```

```
print("=" * 60)
```

```
print("""
```

```
Closest Pair - Brute Force
```

```
-----
```

```
Time Complexity :  $O(n^2)$ 
```

```
Space Complexity :  $O(1)$  auxiliary space
```

Convex Hull - Brute Force

Candidate Edges : $n(n-1)/2$

Time Complexity : $O(n^3)$

Space Complexity : $O(n)$

""")